Format JSON recommandé pour le chargement d’une tournée

Route

GET /api/tournees/jour?dateTournee=2026-04-28&codeTournee=2001&codeLivreur=2

Rôle de la route

Cette route permet à l’application mobile de charger la tournée du jour.

Le livreur :

- s’identifie avec son code livreur ;

- voit son nom affiché ;

- voit la date du jour en lecture seule ;

- choisit uniquement sa tournée ;

- charge les clients ou points de livraison associés.

La date peut être envoyée par l’application, mais elle ne doit pas être modifiable dans l’interface mobile.

JSON recommandé

{

 "schemaVersion": "1.1",

 "dateTournee": "2026-04-28",

 "dateModifiable": false,

 "jourTournee": 2,

 "jourLibelle": "Mardi",

 "codeTournee": "2001",

 "libelleTournee": "MDR VENDEE",

"statutSynchronisation": "NON_ENVOYEE",
"livreur": {
  "codeLivreur": "2",
  "nomLivreur": "DAVID LEBAS"
},
"chargement": {
  "dateGenerationApi": "2026-04-28T07:30:00+02:00",
  "nombrePointsEnvoyes": 8
},
"articlesSaisissables": [
  {
    "codeArticle": "ROLLS",
    "libelle": "Rolls"
  },
  {
    "codeArticle": "TAPIS",
    "libelle": "Tapis"
  },
  {
    "codeArticle": "SACS",
    "libelle": "Sacs"
  }
],
"lignes": [
  {
    "idLigneSource": "2026-04-28|2001|2|1058|1|1",

"ordreArret": 1,
"horaire": 1,
"client": {
  "numClient": "1058",
  "nomClient": "EHPAD L EQUAIZIERE",
  "nomAffiche": "EHPAD EQUAIZIERE GARNACHE"
},
"pointLivraison": {
  "codePDL": "1",
  "descriptionPDL": "EHPAD EQUAIZIERE GARNACHE",
  "adresseLigne1": "7 RUE JAN ET JOEL MARTEL",
  "adresseLigne2": null,
  "adresseLigne3": "-",
  "ville": "LA GARNACHE",
  "codePostal": "85710"
},
"tournee": {
  "codeTournee": "2001",
  "libelleTournee": "MDR VENDEE",
  "jourTournee": 2,
  "schemaLivraison": "1W1"
},
"retour": {
  "jourTourneeRetour": 2,
  "jourRetourLibelle": "Mardi",
  "codeTourneeRetour": "2001",

"libelleTourneeRetour": "MDR VENDEE"
},
"infosLivreur": {
"instructions": null,
"commentaireFiche": null,
"zoneDechargement": "EHPAD",
"zone": "EHPAD",
"precision": null,
"cle": null,
"estFerme": false,
"dateFermeture": null,
"motifFermeture": null
},
"saisie": {
"statutPassage": "A_FAIRE",
"commentaireLivreur": null,
"precisionLivreur": null,
"heureValidation": null,
"estValidee": false,
"quantites": [
{
"codeArticle": "ROLLS",
"libelle": "Rolls",
"quantiteLivree": 0,
"quantiteRecuperee": 0
},

```
        {
          "codeArticle": "TAPIS",
          "libelle": "Tapis",
          "quantiteLivree": 0,
          "quantiteRecuperee": 0
        },
        {
          "codeArticle": "SACS",
          "libelle": "Sacs",
          "quantiteLivree": 0,
          "quantiteRecuperee": 0
        }
      ]
    }
  }
]
}
```


4. Explication de `idLigneSource`

Le champ `idLigneSource` sert à identifier une ligne de tournée de manière stable, même hors connexion.

Il peut être construit par l'API avec les informations suivantes :

2026-04-28|2001|2|1058|1|1

Cela signifie :

Date tournée : 2026-04-28

Tournée : 2001

Jour : 2

Client : 1058

PDL : 1

Arrêt : 1

Exemple recommandé :

"idLigneSource": "2026-04-28|2001|2|1058|1|1"

Ce champ est recommandé pour le stockage local mobile, car il permet de retrouver précisément une ligne même si l’application fonctionne hors connexion pendant la journée.

5. Gestion des dates et heures

Pour ce projet, le plus simple est de rester en heure locale française avec offset.

Exemple :

"2026-04-28T07:30:00+02:00"

Cela évite les ambiguïtés si l’application tourne sur plusieurs appareils.

Les champs concernés sont notamment :

dateGenerationApi

dateChargementMobile

dateEnvoiMobile

heureValidation

dateReceptionApi

6. Format JSON recommandé pour l’envoi du soir

Route

POST /api/synchronisations

Rôle de la route

Cette route reçoit les données saisies pendant la tournée.

Elle doit :

- recevoir les informations générales de la tournée ;

- recevoir le code livreur ;

- recevoir les informations du téléphone ;

- recevoir les lignes saisies ;

- recevoir les quantités livrées et récupérées ;

- contrôler les données ;

- empêcher les doubles envois ;

- enregistrer les données dans SQL Server ;

- verrouiller la tournée côté mobile après succès.

JSON recommandé


```
{
 "schemaVersion": "1.1",
 "idSynchronisation": "4e17a871-5fc5-4f49-8d01-4d791a6d9941",
 "dateTournee": "2026-04-28",
 "codeTournee": "2001",
 "libelleTournee": "MDR VENDEE",
 "livreur": {
   "codeLivreur": "2",
   "nomLivreur": "DAVID LEBAS"
```

},
"mobile": {
"nomAppareil": "Samsung A15",
"versionApplication": "1.0.0",
"dateChargementMobile": "2026-04-28T07:30:00+02:00",
"dateEnvoiMobile": "2026-04-28T16:45:00+02:00"
},
"commentaireGlobal": null,
"lignes": [
{
"idLigneSource": "2026-04-28|2001|2|1058|1|1",
"ordreArret": 1,
"client": {
"numClient": "1058",
"nomClient": "EHPAD L EQUAIZIERE",
"nomAffiche": "EHPAD EQUAIZIERE GARNACHE"
},
"pointLivraison": {
"codePDL": "1",
"descriptionPDL": "EHPAD EQUAIZIERE GARNACHE"
},
"saisie": {
"precisionLivreur": "2 rolls repris au local arrière",
"statutPassage": "FAIT",
"commentaireLivreur": null,
"heureValidation": "2026-04-28T09:12:00+02:00",

"estValidee": true,
"quantites": [
{
"codeArticle": "ROLLS",
"libelle": "Rolls",
"quantiteLivree": 3,
"quantiteRecuperee": 2
},
{
"codeArticle": "TAPIS",
"libelle": "Tapis",
"quantiteLivree": 1,
"quantiteRecuperee": 0
},
{
"codeArticle": "SACS",
"libelle": "Sacs",
"quantiteLivree": 0,
"quantiteRecuperee": 0
}
]
}
}
]
}

7. Pourquoi utiliser un tableau `quantites`

Ancien format moins évolutif :

```
{
 "nbRolls": 3,
 "nbTapis": 1,
 "nbSacs": 0,
 "nbRecuperes": 2
}
```

Ce format fonctionne, mais il oblige à modifier le JSON, les DTO C#, la base SQL et l'application mobile dès qu'un nouvel article est ajouté.

Format recommandé :

```
"quantites": [
 {
   "codeArticle": "ROLLS",
   "libelle": "Rolls",
   "quantiteLivree": 3,
   "quantiteRecuperee": 2
 }
]
```

Ce format est plus évolutif.

Il permet d'ajouter plus tard :

```
{
 "codeArticle": "VETEMENTS",
 "libelle": "Vêtements",
 "quantiteLivree": 0,
```

"quantiteRecuperee": 0

}

sans refaire tout le contrat JSON.

8. Articles saisissables

Pour la première version, les articles visibles dans l'application mobile sont :

ROLLS

TAPIS

SACS

Ils doivent être affichés dans l'écran détail sous forme de deux colonnes :

Article	Livré	Récupéré
Rolls	0	0
Tapis	0	0
Sacs	0	0

Les valeurs doivent être des entiers positifs ou nuls.

quantiteLivree >= 0

quantiteRecuperee >= 0

Les quantités négatives sont interdites.

Les articles doivent idéalement être récupérés depuis les vues ou données existantes liées à ABSSolute. La vue exacte doit encore être confirmée. À défaut, l'API pourra définir une liste d'articles par défaut pour la première version.

9. État actuel du code

Point important

Le code actuel peut encore utiliser l'ancien format avec :

NbExpes

NbRolls

NbVetements

NbTapis

NbSacs

NbRecuperes

Cependant, le nouveau cahier des charges recommande une structure plus évolutive avec :

Mobile_TourneeLigneQuantite

et un tableau JSON :

"quantites": []

passer au nouveau format recommandé

Cette option est plus propre et plus cohérente avec la nouvelle maquette.

Le JSON d’envoi utilise :


```
"saisie": {
 "precisionLivreur": "2 rolls repris au local arrière",
 "statutPassage": "FAIT",
 "commentaireLivreur": null,
 "heureValidation": "2026-04-28T09:12:00+02:00",
 "estValidee": true,
 "quantites": [
   {
     "codeArticle": "ROLLS",
     "libelle": "Rolls",
     "quantiteLivree": 3,
     "quantiteRecuperee": 2
   },
   {
```

"codeArticle": "TAPIS",
"libelle": "Tapis",
"quantiteLivree": 1,
"quantiteRecuperee": 0
},
{
"codeArticle": "SACS",
"libelle": "Sacs",
"quantiteLivree": 0,
"quantiteRecuperee": 0
}
]
}

Avantage :

- plus propre ;
- plus évolutif ;
- cohérent avec les colonnes Livré / Récupéré ;
- compatible avec l’ajout futur de vêtements, draps, serviettes ou autres articles.

Inconvénient :

- nécessite de modifier les DTO C# ;
- nécessite de modifier SynchronisationsRepository.cs ;
- nécessite de modifier ou ajouter la table Mobile_TourneeLigneQuantite ;
- nécessite de refaire les tests POST.

Recommandation

Pour le projet final, il est recommandé de passer au nouveau format avec :

schemaVersion = 1.1

quantites[]

Mobile_TourneeLigneQuantite

Ce format correspond mieux à la maquette corrigée et au besoin métier confirmé.

10. Correspondance avec les tables SQL

Table `Mobile_Livreur`

Le bloc suivant alimente la table `Mobile_Livreur` :

```
"livreur": {
 "codeLivreur": "2",
 "nomLivreur": "DAVID LEBAS"
}
```

Le repository peut utiliser un `MERGE` :

- si le livreur existe déjà avec le même CodeLivreur, son nom est mis à jour ;
- sinon, le livreur est créé.

Le code livreur est obligatoire.

Table `Mobile_Tournee`

Les informations générales de la tournée sont enregistrées dans `Mobile_Tournee`.

Champs à remplir :

IdSynchronisation

DateTournee

CodeTournee

LibelleTournee

IdLivreur

StatutSynchronisation = ENVOYEE

DateChargementMobile

DateReceptionApi

DateEnvoi

EstVerrouillee = 1

NombrePointsPrevus

NombrePointsSaisis

CommentaireGlobal

NomAppareil

VersionApplication

AdresseIP

DateCreation

DateModification

Exemple attendu en SQL :

DateTournee = 2026-04-28

CodeTournee = 2001

LibelleTournee = MDR VENDEE

StatutSynchronisation = ENVOYEE

NomAppareil = Samsung A15

VersionApplication = 1.0.0

EstVerrouillee = 1

Table `Mobile_TourneeLigne`

Chaque élément du tableau `lignes` est enregistré dans `Mobile_TourneeLigne`.

Champs recommandés :

IdTourneeMobile

IdLigneSource

CodeTournee

OrdreArret

NumClient

NomClient

NomAffiche

CodePDL

DescriptionPDL

PrecisionLivreur

StatutPassage

CommentaireLivreur

HeureValidation

EstValidee

DateCreation

DateModification

Champs complémentaires possibles :

AdresseLigne1

AdresseLigne2

AdresseLigne3

Ville

CodePostal

JourTournee

SchemaLivraison

LibelleTournee

JourTourneeRetour

CodeTourneeRetour

LibelleTourneeRetour

Instructions

ZoneDechargement

Zone

TypeLinge

EstFerme

DateFermeture

MotifFermeture

Table `Mobile_TourneeLigneQuantite`

Cette table est recommandée pour gérer les colonnes Livré / Récupéré par article.

Elle permet d’éviter d’ajouter une nouvelle colonne SQL à chaque nouvel article.

Champs recommandés :

IdQuantite

IdTourneeLigne

CodeArticle

LibelleArticle

QuantiteLivree

QuantiteRecuperee

DateCreation

DateModification

Exemple :

IdTourneeLigne = 15

CodeArticle = ROLLS

LibelleArticle = Rolls

QuantiteLivree = 3

QuantiteRecuperee = 2

Autre exemple :

IdTourneeLigne = 15

CodeArticle = TAPIS

LibelleArticle = Tapis

QuantiteLivree = 1

QuantiteRecuperee = 0

Table `Mobile_LogSynchronisation`

Après une synchronisation réussie, l'API insère un log dans `Mobile_LogSynchronisation`.

Champs remplis :

IdTourneeMobile

IdLivreur

IdSynchronisation

DateEvenement

TypeEvenement

Niveau

Message

DetailTechnique

AdresseIP

NomAppareil

VersionApplication

Exemple attendu :

TypeEvenement = ENVOI_REUSSI

Niveau = INFO

Message = Synchronisation enregistrée avec succès.

DetailTechnique = Tournée 2001 du 2026-04-28 synchronisée avec 1 ligne(s).

NomAppareil = Samsung A15

VersionApplication = 1.0.0

11. Calcul des quantités

Avec le nouveau format, le calcul se fait à partir du tableau `quantites`.

Exemple JSON :


```
"quantites": [
 {
   "codeArticle": "ROLLS",
   "libelle": "Rolls",
   "quantiteLivree": 3,
   "quantiteRecuperee": 2
 },
 {
   "codeArticle": "TAPIS",
   "libelle": "Tapis",
   "quantiteLivree": 1,
   "quantiteRecuperee": 0
 },
 {
   "codeArticle": "SACS",
   "libelle": "Sacs",
```

"quantiteLivree": 0,

"quantiteRecuperee": 0

}

]

Calcul possible :

TotalLivré = somme des quantiteLivree

TotalRécupéré = somme des quantiteRecuperee

Résultat :

TotalLivré = 4

TotalRécupéré = 2

Si les anciennes colonnes de compatibilité sont conservées dans `Mobile_TourneeLigne`, elles peuvent être calculées automatiquement :

QuantiteLivree = somme des quantiteLivree

QuantiteReprise = somme des quantiteRecuperee

12. Règles métier validées ou à valider

Règles générales

L'API doit refuser :

- une requête sans schemaVersion ;

- une schemaVersion non supportée ;

- une requête sans idSynchronisation ;

- une requête sans dateTournee ;

- une requête sans codeTournee ;

- une requête sans livreur.codeLivreur ;

- une requête sans lignes ;

- une ligne sans idLigneSource ;

- une ligne sans statutPassage ;

- une ligne validée sans heureValidation ;

- une ligne avec des quantités négatives ;

- une ligne avec des quantités non entières ;

- une ligne avec statut NON_FAIT sans commentaire ;

- une ligne avec statut ANOMALIE sans commentaire ;

- une synchronisation contenant encore une ligne A_FAIRE ;

- un double envoi avec le même idSynchronisation ;

- un double envoi pour la même date, la même tournée et le même livreur.

Statuts autorisés

Les statuts autorisés sont :

A_FAIRE

FAIT

NON_FAIT

ANOMALIE

Dans le chargement du matin, les lignes peuvent être en statut :

A_FAIRE

Dans l'envoi final du soir, le statut `A_FAIRE` doit être refusé.

Quantités négatives interdites

L'API doit refuser :

quantiteLivree < 0

quantiteRecuperee < 0

Exemple de réponse :

{

"statut": "VALIDATION_ERROR",

"errors": [

"Ligne 1, article ROLLS : QuantiteLivree ne peut pas être négative."

]

}

Statut `FAIT`

Si :

"statutPassage": "FAIT"

Alors :

estValidee = true

heureValidation obligatoire

commentaireLivreur facultatif

Statut `NON_FAIT`

Si :

"statutPassage": "NON_FAIT"

Alors :

estValidee = true

heureValidation obligatoire

commentaireLivreur obligatoire

Exemple de réponse :

{

"statut": "VALIDATION_ERROR",

"errors": [

"Ligne 1 : CommentaireLivreur est obligatoire pour le statut NON_FAIT."

]

}

Statut `ANOMALIE`

Si :

"statutPassage": "ANOMALIE"

Alors :

estValidee = true

heureValidation obligatoire

commentaireLivreur obligatoire

Exemple de réponse :

{

"statut": "VALIDATION_ERROR",

"errors": [

"Ligne 1 : CommentaireLivreur est obligatoire pour le statut ANOMALIE."

]

}

Ligne validée sans heure

Si :

"estValidee": true

Alors :

heureValidation obligatoire

Exemple de réponse :

```
{
 "statut": "VALIDATION_ERROR",
 "errors": [
   "Ligne 1 : HeureValidation est obligatoire."
 ]
}
```

Ligne non validée dans l’envoi final

Dans :

POST /api/synchronisations

une ligne avec :

"estValidee": false

doit être refusée.

Objectif :

éviter qu’une tournée soit envoyée avec des lignes incomplètes.

13. Résultats de tests à conserver

Test	Fichier JSON	Résultat attendu
Synchronisation valide	`sync-valide.json`	`200 OK / SUCCESS`
Doublon `idSynchronisation`	`sync-doublon.json`	`409 Conflict / CONFLICT`

Double envoi même tournée	`sync-double-envoi-tournee.json`	`409 Conflict / CONFLICT`
Quantité négative	`sync-quantite-negative.json`	`400 Bad Request / VALIDATION_ERROR`
`NON_FAIT` sans commentaire	`sync-non-fait-sans-commentaire.json`	`400 Bad Request / VALIDATION_ERROR`
`ANOMALIE` sans commentaire	`sync-anomalie-sans-commentaire.json`	`400 Bad Request / VALIDATION_ERROR`
Ligne validée sans `heureValidation`	`sync-validee-sans-heure.json`	`400 Bad Request / VALIDATION_ERROR`
`estValidee = false`	`sync-est-validee-false.json`	`400 Bad Request / VALIDATION_ERROR`
Statut `A_FAIRE` dans envoi final	`sync-a-faire.json`	`400 Bad Request / VALIDATION_ERROR`
`idLigneSource` dupliqué dans la requête	`sync-idligne-dupliqué.json`	`400 Bad Request / VALIDATION_ERROR`

14. Vérifications SQL à réaliser après un test valide

Après un test valide, vérifier les tables suivantes :

Mobile_Tournee

Mobile_TourneeLigne

Mobile_TourneeLigneQuantite

Mobile_LogSynchronisation

Résultat attendu dans `Mobile_Tournee`

- tournée insérée ;

- statut ENVOYEE ;

- verrouillage actif ;

- DateEnvoi renseignée ;

- IdSynchronisation renseigné ;

- IdLivreur associé.

Résultat attendu dans `Mobile_TourneeLigne`

- lignes insérées ;

- IdLigneSource enregistré ;

- client enregistré ;

- point de livraison enregistré ;

- statut du passage enregistré ;

- commentaire enregistré si nécessaire ;

- heureValidation enregistrée ;

- EstValidee enregistré.

Résultat attendu dans `Mobile_TourneeLigneQuantite`

- une ligne insérée par article ;

- CodeArticle enregistré ;

- LibelleArticle enregistré ;

- QuantiteLivree enregistrée ;

- QuantiteRecuperee enregistrée.

Résultat attendu dans `Mobile_LogSynchronisation`

- log ENVOI_REUSSI créé ;

- niveau INFO ;

- message de succès enregistré ;

- détail technique enregistré ;

- nom de l’appareil enregistré ;

- version de l’application enregistrée.

---

15. État actuel du backend

État fonctionnel actuel à maintenir ou à adapter selon la version choisie :

GET /api/tournees/jour : utilisé pour charger la tournée du jour.

POST /api/synchronisations : utilisé pour recevoir l’envoi du soir.

Anti-doublon idSynchronisation : à conserver.

Anti-doublon DateTournee + CodeTournee + IdLivreur : à conserver.

Validations métier principales : à conserver.

Écriture SQL : à adapter si passage à Mobile_TourneeLigneQuantite.

Logging SQL : à conserver.

16. Index SQL importants

Les index suivants doivent exister sur `Mobile_Tournee`.

Index anti-double envoi métier

UX_Mobile_Tournee_EnvoiUnique

Colonnes : DateTournee, CodeTournee, IdLivreur

Rôle : empêche une même tournée d’être envoyée deux fois par le même livreur.

Index anti-doublon technique

UQ_Mobile_Tournee_IdSynchronisation

Colonne : IdSynchronisation

Rôle : empêche le même paquet de synchronisation mobile d’être renvoyé deux fois.